LEC - 59
SUMMARY
NOTES

```
import numpy as np
```

```
# A factory collects temperatures from 20 sensor readings:

temperature = np.array([
    28.5, 31.2, 29.8, 35.6, 38.1,
    40.2, 27.9, 30.5, 42.3, 39.8,
    36.4, 33.2, 29.1, 41.7, 44.2,
    31.8, 34.5, 37.6, 26.9, 43.1
])
```

```
Calculate mean, median, minimum and maximum.

Calculate variance and standard deviation.

Find all temperatures more than one standard deviation above the mean.

Calculate the absolute difference between every temperature and the mean.

Find the five readings having the largest deviation from the mean.

Round all temperatures to the nearest whole number.
```

```
temperature = np.array([
    28.5, 31.2, 29.8, 35.6, 38.1,
    40.2, 27.9, 30.5, 42.3, 39.8,
    36.4, 33.2, 29.1, 41.7, 44.2,
    31.8, 34.5, 37.6, 26.9, 43.1
])
```

```
print(temperature)
```

```
[28.5 31.2 29.8 35.6 38.1 40.2 27.9 30.5 42.3 39.8 36.4 33.2 29.1 41.7
 44.2 31.8 34.5 37.6 26.9 43.1]
```

```
temperature.ndim
```

```
1
```

```
temperature.shape
```

```
(20,)
```

```
# Calculate mean, median, minimum and maximum.
avg_t = np.mean(temperature)
avg_t
```

```
np.float64(35.12)
```

```
median_t = np.median(temperature)
median_t
```

```
np.float64(35.05)
```

```
mnt = np.min(temperature)
mnt
```

```
np.float64(26.9)
```

```
mxt = np.max(temperature)
mxt
```

```
np.float64(44.2)
```

```
stdt= np.std(temperature)
stdt
```

```
np.float64(5.410415880503088)
```

```
var = np.square(stdt)
var
```

```
np.float64(29.2726)
```

```
# Find all temperatures more than one standard deviation above the mean.
new_t = temperature[ temperature > (avg_t + stdt)]
new_t
```

```
array([42.3, 41.7, 44.2, 43.1])
```

```
np.where( temperature > (avg_t + stdt), "Yes", "No")
```

```
array(['No', 'No', 'No', 'No', 'No', 'No', 'No', 'No', 'Yes', 'No', 'No',
       'No', 'No', 'Yes', 'Yes', 'No', 'No', 'No', 'No', 'Yes'],
      dtype='<U3')
```

```
temperature
```

```
array([28.5, 31.2, 29.8, 35.6, 38.1, 40.2, 27.9, 30.5, 42.3, 39.8, 36.4,
       33.2, 29.1, 41.7, 44.2, 31.8, 34.5, 37.6, 26.9, 43.1])
```

```
# Calculate the absolute difference between every temperature and the mean.

deviation = np.abs(temperature - avg_t)
deviation
```

```
array([6.62, 3.92, 5.32, 0.48, 2.98, 5.08, 7.22, 4.62, 7.18, 4.68, 1.28,
       1.92, 6.02, 6.58, 9.08, 3.32, 0.62, 2.48, 8.22, 7.98])
```

```
# Find the five readings having the largest deviation from the mean.
largest_deviation = np.sort(deviation)
```

```
largest_deviation
```

```
array([0.48, 0.62, 1.28, 1.92, 2.48, 2.98, 3.32, 3.92, 4.62, 4.68, 5.08,
       5.32, 6.02, 6.58, 6.62, 7.18, 7.22, 7.98, 8.22, 9.08])
```

```
large5 = largest_deviation[-5:]
large5
```

```
array([7.18, 7.22, 7.98, 8.22, 9.08])
```

```
# Round all temperatures to the nearest whole number.
temperature
```

```
array([28.5, 31.2, 29.8, 35.6, 38.1, 40.2, 27.9, 30.5, 42.3, 39.8, 36.4,
       33.2, 29.1, 41.7, 44.2, 31.8, 34.5, 37.6, 26.9, 43.1])
```

```
np.round(temperature)
```

```
array([28., 31., 30., 36., 38., 40., 28., 30., 42., 40., 36., 33., 29.,
       42., 44., 32., 34., 38., 27., 43.])
```

```
Categorize temperatures:
>= 40 → "Critical"
>= 35 → "Warning"
Otherwise → "Normal"

Find the number and percentage of critical readings.

Reshape the readings into a 4 × 5 array.

Find the average temperature for each row.
```

```
cat_t = np.where(temperature >= 40, "Critical", np.where(temperature >= 35, "Warning", "Normal"))
cat_t
```

```
array(['Normal', 'Normal', 'Normal', 'Warning', 'Warning', 'Critical',
       'Normal', 'Normal', 'Critical', 'Warning', 'Warning', 'Normal',
       'Normal', 'Critical', 'Critical', 'Normal', 'Normal', 'Warning',
       'Normal', 'Critical'], dtype='<U8')
```

```
temperature
```

```
array([28.5, 31.2, 29.8, 35.6, 38.1, 40.2, 27.9, 30.5, 42.3, 39.8, 36.4,
       33.2, 29.1, 41.7, 44.2, 31.8, 34.5, 37.6, 26.9, 43.1])
```

```
critical_t = cat_t[ cat_t == "Critical"]
```

```
c = len(critical_t)
c
```

```
5
```

```
n = len(cat_t)
n
```

```
20
```

```
perc = c*100/n
perc
```

```
25.0
```

```
# Reshape the readings into a 4 × 5 array.
new_t = temperature.reshape(4,5)
new_t
```

```
array([[28.5, 31.2, 29.8, 35.6, 38.1],
       [40.2, 27.9, 30.5, 42.3, 39.8],
       [36.4, 33.2, 29.1, 41.7, 44.2],
       [31.8, 34.5, 37.6, 26.9, 43.1]])
```

```
# Find the average temperature for each row.

np.mean(new_t , axis = 1 )
```

```
array([32.64, 36.14, 36.92, 34.78])
```

```
Consider the following employee data:

employees = np.array([
    [101, 42000, 3, 450000],
    [102, 65000, 6, 720000],
    [103, 38000, 2, 300000],
    [104, 90000, 8, 950000],
    [105, 55000, 5, 610000],
    [106, 72000, 7, 830000]
])


Columns represent:

Employee_ID
Salary
Experience
Annual_Sales
```

```
Extract salary, experience and annual sales.

Calculate sales-to-salary ratio.

Find the employee with the highest annual sales.

Find employees having at least 5 years experience AND sales above ₹600,000.

Calculate bonus:
Sales >= ₹800,000 → 10%
Sales >= ₹600,000 → 7%
Otherwise → 3%
```

```
employees = np.array([
    [101, 42000, 3, 450000],
    [102, 65000, 6, 720000],
    [103, 38000, 2, 300000],
    [104, 90000, 8, 950000],
    [105, 55000, 5, 610000],
    [106, 72000, 7, 830000]
])
```

```
# Extract salary, experience and annual sales.

salary = employees[:,1]
experience = employees[:,2]
sales = employees[:,3]
```

```
salary
```

```
array([42000, 65000, 38000, 90000, 55000, 72000])
```

```
experience
```

```
array([3, 6, 2, 8, 5, 7])
```

```
sales
```

```
array([450000, 720000, 300000, 950000, 610000, 830000])
```

```
# Calculate sales-to-salary ratio.

sts = np.round(sales/salary)
sts
```

```
array([11., 11.,  8., 11., 11., 12.])
```

```
# Find the employee with the highest annual sales.
np.argmax(sales)
```

```
np.int64(3)
```

```
employees[np.argmax(sales)]
```

```
array([   104,  90000,      8, 950000])
```

```
# Find employees having at least 5 years experience AND sales above ₹600,000.
criteria = (experience >= 5) & (sales > 600000)
employees[criteria]
```

```
array([[   102,  65000,      6, 720000],
       [   104,  90000,      8, 950000],
       [   105,  55000,      5, 610000],
       [   106,  72000,      7, 830000]])
```

```
Calculate bonus:
Sales >= ₹800,000 → 10%
Sales >= ₹600,000 → 7%
Otherwise → 3%
```

```
bonus = np.where(sales>= 800000, 0.1*sales, np.where(sales >= 600000, 0.07*sales, 0.03*sales))
bonus
```

```
array([13500., 50400.,  9000., 95000., 42700., 83000.])
```

```
Calculate total bonus amount.

Find average salary of employees receiving a 10% bonus.

Find median salary.

Find employees whose salary is above median salary but experience is less than 6 years.
```

```
Round sales-to-salary ratio to 2 decimal places.
```

```
# Calculate total bonus amount.

total_bonus = np.sum(bonus)
total_bonus
```

```
np.float64(293600.0)
```

```
# Find average salary of employees receiving a 10% bonus.
sales>=800000
```

```
array([False, False, False,  True, False,  True])
```

```
s = salary[ sales >= 800000 ]
```

```
np.mean(s)
```

```
np.float64(81000.0)
```

```
# Find median salary.
np.median(salary)
```

```
np.float64(60000.0)
```

```
median_s = np.median(salary)
median_s
```

```
np.float64(60000.0)
```

```
Start coding or generate with AI.
```

```
# Find employees whose salary is above median salary but experience is less than 6 years.
criteria = (salary > median_s) & (experience < 6)
criteria
```

```
array([False, False, False, False, False, False])
```

```
employees[criteria]
```

```
array([], shape=(0, 4), dtype=int64)
```

```
# Round sales-to-salary ratio to 2 decimal places.

sts
```

```
array([11., 11.,  8., 11., 11., 12.])
```

```
sts2 = np.round(sales/salary, 2)
sts2
```

```
array([10.71, 11.08,  7.89, 10.56, 11.09, 11.53])
```

```
Consider the following 6 × 6 grayscale image:

image = np.array([
    [10, 25, 40, 55, 70, 85],
    [20, 35, 50, 65, 80, 95],
    [30, 45, 60, 75, 90, 105],
    [40, 55, 70, 85, 100, 115],
    [50, 65, 80, 95, 110, 125],
    [60, 75, 90, 105, 120, 135]
])
```

```
Find ndim, shape and size.

Extract the central 4 × 4 region.

Increase the central region by 20%.
```

```
Cap all pixel values above 100 at 100.

Find mean, minimum and maximum pixel values.
```

```
image = np.array([
    [10, 25, 40, 55, 70, 85],
    [20, 35, 50, 65, 80, 95],
    [30, 45, 60, 75, 90, 105],
    [40, 55, 70, 85, 100, 115],
    [50, 65, 80, 95, 110, 125],
    [60, 75, 90, 105, 120, 135]
])
```

```
# Find ndim, shape and size.
image.ndim
```

```
2
```

```
image.shape
```

```
(6, 6)
```

```
image.size
```

```
36
```

```
image
```

```
array([[ 10,  25,  40,  55,  70,  85],
       [ 20,  35,  50,  65,  80,  95],
       [ 30,  45,  60,  75,  90, 105],
       [ 40,  55,  70,  85, 100, 115],
       [ 50,  65,  80,  95, 110, 125],
       [ 60,  75,  90, 105, 120, 135]])
```

```
# Extract the central 4 × 4 region.
center = image[1:5, 1:5]
center
```

```
array([[ 35,  50,  65,  80],
       [ 45,  60,  75,  90],
       [ 55,  70,  85, 100],
       [ 65,  80,  95, 110]])
```

```
# Increase the central region by 20%.
image[1:5,1:5] = image[1:5,1:5]*1.2
```

```
image
```

```
array([[ 10,  25,  40,  55,  70,  85],
       [ 20,  42,  60,  78,  96,  95],
       [ 30,  54,  72,  90, 108, 105],
       [ 40,  66,  84, 102, 120, 115],
       [ 50,  78,  96, 114, 132, 125],
       [ 60,  75,  90, 105, 120, 135]])
```

```
# Cap all pixel values above 100 at 100.
image = np.where(image > 100, 100, image)
image
```

```
array([[ 10,  25,  40,  55,  70,  85],
       [ 20,  42,  60,  78,  96,  95],
       [ 30,  54,  72,  90, 100, 100],
       [ 40,  66,  84, 100, 100, 100],
       [ 50,  78,  96, 100, 100, 100],
       [ 60,  75,  90, 100, 100, 100]])
```

```
# Find mean, minimum and maximum pixel values.

np.mean(image)
```

```
np.float64(73.91666666666667)
```

```
np.min(image)
```

```
np.int64(10)
```

```
np.max(image)
```

```
np.int64(100)
```

```
Find pixels above the 75th percentile.

Find pixels below the overall mean.

Flatten the image.

Reshape the flattened array back into 6 × 6.
```

```
# Find pixels above the 75th percentile.

p75 = np.percentile(image, 75)
p75
```

```
np.float64(100.0)
```

```
image[image > p75]
```

```
array([], dtype=int64)
```

```
avg = np.mean(image)
```

```
image[image < avg]
```

```
array([10, 25, 40, 55, 70, 20, 42, 60, 30, 54, 72, 40, 66, 50, 60])
```

```
image.flatten()
```

```
array([ 10,  25,  40,  55,  70,  85,  20,  42,  60,  78,  96,  95,  30,
        54,  72,  90, 100, 100,  40,  66,  84, 100, 100, 100,  50,  78,
        96, 100, 100, 100,  60,  75,  90, 100, 100, 100])
```

```
new_image = image.reshape(6,6)
new_image
```

```
array([[ 10,  25,  40,  55,  70,  85],
       [ 20,  42,  60,  78,  96,  95],
       [ 30,  54,  72,  90, 100, 100],
       [ 40,  66,  84, 100, 100, 100],
       [ 50,  78,  96, 100, 100, 100],
       [ 60,  75,  90, 100, 100, 100]])
```

```
Start coding or generate with AI.
```